

Lab 2: Layered Architecture Design (Logical View)

BadNet - Badminton Social Network

Introduction

This report presents the logical view and layered architecture design of the **BadNet** (Badminton Social Network) system. Building upon the functional and non-functional requirements identified in Lab 1, this stage of the design emphasizes the system's static structure and internal organization rather than runtime behavior.

The primary objective of this architecture is to clearly separate responsibilities across different parts of the system, thereby improving maintainability, scalability, and long-term extensibility. The system adopts a **strict layered architecture pattern**, where each layer has a well-defined role and communicates only with its adjacent lower layer. This approach minimizes coupling between components and allows changes in one layer to have minimal impact on others.

Activity Practice 1: Defining Layers and Responsibilities

1.1 The Four Main Architectural Layers

The backend of the BadNet system is structured into four logical layers:

Layer	Purpose/Responsibility	Output/Artifact
1. Presentation Layer (UI/API)	Handles incoming HTTP requests, routing, input validation, and returns responses (Blade views or JSON). Serves as the entry point for users.	Controllers (e.g., MatchController, ChallengeController)
2. Business Logic Layer (Service)	Encapsulates core business rules such as ELO calculation, match settlement, challenge validation, betting logic, and tournament management.	Services (e.g., EloService, MatchService)
3. Persistence Layer (Data Access)	Abstracts database operations using Eloquent ORM. Responsible for querying and mapping data to domain models.	Eloquent Models / Repositories (e.g., Match, Challenge)
4. Data Layer (Physical storage)	The physical database (MySQL) that stores all persistent information: users, matches, challenges, teams, tournaments, posts, etc.	Database Schema (MySQL Tables)

1.2 Data Flow Logic

Scenario: A user views the details of a specific match.

The interaction begins when the user (through Blade frontend) sends a request to the endpoint `/matches/{id}`.

1. **Presentation Layer** (MatchController) receives the request, extracts the match ID, and forwards it to the Business Logic Layer.
2. **Business Logic Layer** (MatchService) performs validations (e.g., match exists, status check) and business operations (e.g., retrieve related bets, participants, ELO impact).
3. **Persistence Layer** uses Eloquent to query the database.
4. **Data Layer** (MySQL) returns the raw data.
5. Data flows back through the layers. The controller ultimately returns the data to the Blade view or as JSON.

This clear and linear flow ensures that each layer remains focused on its specific responsibility.

Activity Practice 2: Component Identification (Match Management)

Focusing on the “**View Match Details**” functionality (a core feature of BadNet).

2.1 Component Identification

Layer 1 (Presentation):

- **Component Name:** MatchController
- **Responsibility:** Receives HTTP requests (GET `/matches/{id}` or `/challenges/{id}`), extracts parameters, calls the Service layer, and returns the appropriate view or JSON response.

Layer 2 (Business Logic):

- **Component Name:** MatchService (hoặc EloService kết hợp)
- **Responsibility:** Implements business logic such as calculating ELO changes, validating match result, checking challenge rules, aggregating betting data, and determining match status.

Layer 3 (Persistence):

- **Component Name:** Match Model (Eloquent) / MatchRepository (nếu có)
- **Responsibility:** Executes queries to the matches table, handles relationships (with users, teams, tournaments), and returns domain models.

2.2 Interface Definitions

MatchService Interface (for Layer 1):

- **Input:** Match ID
- **Output:** Processed match data with business information

Match Model / Repository Interface (for Layer 2):

- **Input:** Match ID (primary key)
- **Output:** Match model instance or null

Activity Practice 3: Component Diagram Modeling

